



PL-300

Search

CTRL

K



Modelloptimierung

Enders Training

CONTENTS

Grundprinzip: Kleinere Modelle, bessere Leistung

Leistung messen: Der Performance Analyzer

Ergebnisse interpretieren

Was bei langen Ladezeiten prüfen

Metadaten analysieren in Power Query

Automatisches Datum/Uhrzeit deaktivieren

Variablen für Leistung und Debugging

Leistungsverbesserung durch Variablen

Weitere Vorteile von Variablen

Kardinalität reduzieren

Beziehungskardinalität

Daten zusammenfassen statt granular importieren

DirectQuery-Modelle optimieren

Wann DirectQuery sinnvoll ist

Verhalten von DirectQuery-Verbindungen

Einschränkungen

Optimierungsmaßnahmen

Aggregationen erstellen und verwalten

Wann Aggregationen sinnvoll sind

Aggregationen in Power Query erstellen

Aggregationen verwalten

Zusammenfassung

Modul 3 · Skript 15

Modelloptimierung

Leistungsanalyse, Variablen, Kardinalität, DirectQuery und Aggregationen

Ein schlecht optimiertes semantisches Modell ist in neun von zehn Fällen die Ursache für schlechte Berichtsleistung — zu lange Ladezeiten, träge Filter, überlastete Kapazitäten. Dieses Skript beschreibt die wichtigsten Optimierungstechniken: Wie Engpässe identifiziert werden, wie Variablen die DAX-Leistung verbessern, was Kardinalität mit Leistung zu tun hat, wie DirectQuery-Modelle optimiert werden und wie Aggregationen Modelle drastisch verkleinern können.

Grundprinzip: Kleinere Modelle, bessere Leistung

Ein kleineres semantisches Modell verbraucht weniger Arbeitsspeicher, aktualisiert schneller und liefert Abfrageergebnisse schneller. Der Optimierungsprozess zielt darauf ab, die Modellgröße zu minimieren und die Effizienz der Berechnungen zu maximieren. Konkrete Entwurfsentscheidungen sollten:

- Richtige Datentypen verwenden (numerische Typen statt Text wo möglich)
- Unnötige Spalten und Zeilen entfernen
- Wiederholte Werte vermeiden
- Numerische Spalten als Measures anzeigen statt als aggregierbare Spalten
- Spaltenkardinalität reduzieren
- Modellmetadaten analysieren
- Daten nach Möglichkeit vorab zusammenfassen

Hinweis Für die tiefer gehende Optimierung komplexer Unternehmensmodelle — insbesondere Abfragepläne und Datenbankoptimierung — empfiehlt sich die Zusammenarbeit mit einem Data Engineer. Die in diesem Skript beschriebenen Techniken bilden jedoch eine solide Grundlage.

Leistung messen: Der Performance Analyzer

Das erste Werkzeug für jede Optimierung ist der **Leistungsanalyse**-Bereich in Power BI Desktop (**Ansicht** → **Leistungsanalyse**). Er zeigt für jedes Visual, wie lange drei Kategorien gedauert haben:

Kategorie	Was wird gemessen
DAX-Abfrage	Zeit für Abfrage und Antwort aus der Engine
Visualanzeige	Zeit für das Rendering des Visuals (inkl. Geocodierung, Webbilder)
Andere	Wartezeiten, Abfragevorbereitung, Hintergrundprozesse

Wichtig Vor dem Start der Leistungsanalyse beide Caches leeren:

- **Visueller Cache:** Power BI Desktop schließen, eine leere Seite hinzufügen, speichern und auf der leeren Seite wieder öffnen
 - **Daten-Engine-Cache:** Power BI Desktop neu starten oder DAX Studio → "Cache löschen"
- Ohne geleerte Caches sind die Messwerte nicht aussagekräftig.

Screenshot Dateiname: 2-performance-analyzer-overview-ssm.png — Bereich Leistungsanalyse mit der Schaltfläche "Aufzeichnung starten" und den Ergebnissen nach einer Interaktion.

Ergebnisse interpretieren

Die Ergebnisse nach Gesamtdauer (absteigend) sortieren. DAX-Abfragen über **120 Millisekunden** sind ein guter Ausgangspunkt für die Untersuchung. Mit der Schaltfläche **Abfrage kopieren** lässt sich der DAX-Ausdruck in die Zwischenablage kopieren und in **DAX Studio** für tiefere Analyse einfügen.

Screenshot Dateiname: 2-categories-performance-analyze-results-ss.png — Ergebnisse der Leistungsanalyse mit den drei Kategorien DAX-Abfrage, Visualanzeige und Andere für jedes Visual.

Was bei langen Ladezeiten prüfen

Visuals: Zu viele Visuals auf einer Seite belasten die Leistung. Jedes Visual löst eine oder mehrere Abfragen aus — bei 20 Visuals und 10 gleichzeitigen Nutzern sind das potenziell 200+ Abfragen. Visuals mit mehr als **100 Feldern** laden langsam. Alternative: Drillthrough-Seiten oder Tooltip-Seiten statt zusätzlicher Visuals.

DAX-Abfragen: **FILTER** durch **KEEPFILTERS** ersetzen kann in vielen Fällen die Abfragedauer erheblich reduzieren.

Semantisches Modell: Falsch konfigurierte Beziehungskardinalität, überflüssige Spalten, oder aktiviertes **Autom. Datum/Uhrzeit** können Ursachen sein.

Metadaten analysieren in Power Query

Power Query bietet drei Datenvorschau-Optionen (**Ansicht**-Menüband):

- **Spaltenqualität** — Anteil gültiger, fehlerhafter und leerer Werte
- **Spaltenverteilung** — Häufigkeit und Verteilung der Werte (zeigt Kardinalität)
- **Spaltenprofil** — Statistikdiagramm und Verteilungsdiagramm

Hinweis Standardmäßig analysiert Power Query nur die ersten 1.000 Zeilen. Für das gesamte Dataset die Einstellung am unteren Rand des Fensters auf "*Spaltenprofilierung basierend auf dem gesamten DataSet*" umstellen.

Automatisches Datum/Uhrzeit deaktivieren

Die Option **Autom. Datum/Uhrzeit** erstellt für jede Datums-/Datum-Uhrzeit-Spalte automatisch eine ausgeblendete berechnete Tabelle. Diese Tabellen vergrößern das Modell und verlängern die Aktualisierungszeit. Für professionelle Modelle mit eigener Datumstabelle sollte diese Option deaktiviert werden:

Datei → Optionen und Einstellungen → Optionen → Daten laden → Zeitintelligenz → Autom. Datum/Uhrzeit

Variablen für Leistung und Debugging

Variablen in DAX-Formeln werden nur **einmal** berechnet — unabhängig davon, wie oft sie in der Formel referenziert werden. Das ist der entscheidende Leistungsvorteil.

Leistungsverbesserung durch Variablen

```
-- Ohne Variable – PARALLELPERIOD wird zweimal ausgewertet:
Sales YoY Growth =
DIVIDE(
    ([Sales] - CALCULATE([Sales], PARALLELPERIOD('Date'[Date], -12, MONTH))),
    CALCULATE([Sales], PARALLELPERIOD('Date'[Date], -12, MONTH))
)

-- Mit Variable – PARALLELPERIOD wird einmal ausgewertet:
Sales YoY Growth =
VAR SalesPriorYear =
    CALCULATE([Sales], PARALLELPERIOD('Date'[Date], -12, MONTH))
VAR SalesVariance =
    DIVIDE(([Sales] - SalesPriorYear), SalesPriorYear)
RETURN SalesVariance
```

Bei komplexen Modellen mit vielen solcher Measures kann die Verwendung von Variablen die gesamte Abfrageverarbeitungszeit **halbieren**.

Weitere Vorteile von Variablen

- **Lesbarkeit** — Sprechende Variablenamen (**SalesPriorYear** statt geschachtelter Funktionsaufruf)
- **Wartbarkeit** — Änderungen nur an einer Stelle notwendig
- **Debugging** — RETURN temporär durch eine Variable ersetzen, um deren Zwischenwert zu prüfen:

```
Sales YoY Growth =
VAR SalesPriorYear = CALCULATE([Sales], PARALLELPERIOD('Date'[Date], -12, MONTH))
VAR SalesVariance = DIVIDE(([Sales] - SalesPriorYear), SalesPriorYear)
RETURN SalesPriorYear -- temporär: zeigt nur den Vorjahreswert zum Debuggen
```

- **Reduzierte Komplexität** — Variablen ersetzen die älteren **EARLIER**- und **EARLIEST**-Funktionen, die für verschachtelte Filterkontexte notwendig waren und schwer zu lesen sind.
-

Kardinalität reduzieren

Kardinalität (Spalte) Die Anzahl eindeutiger Werte in einer Spalte. Eine Spalte mit vielen eindeutigen Werten hat hohe Kardinalität (z. B. eine Transaktions-ID). Eine Spalte mit wenigen unterschiedlichen Werten hat niedrige Kardinalität (z. B. Land, Status). Niedrigere Kardinalität → bessere Leistung.

Die **Spaltenverteilung** in Power Query zeigt die Kardinalität jeder Spalte. Spalten mit unnötig hoher Kardinalität (z. B. eine ID-Spalte, die nirgends in Beziehungen oder Berichten verwendet wird) sollten entfernt werden.

Beziehungskardinalität

Bei Modellbeziehungen beschreibt Kardinalität die Richtung der Beziehung (n:1, 1:1, 1:n, n:n). Fehlerhaft konfigurierte Kardinalität — z. B. eine 1-Seite als n-Seite — führt zu falschen Abfrageergebnissen und Leistungsproblemen. Beide Beziehungsspalten müssen **denselben Datentyp** haben; Integer-Spalten funktionieren besser als Text.

Daten zusammenfassen statt granular importieren

Die wirksamste Strategie zur Kardinalitätsreduktion ist die **Voraggregation auf Quellebene**. Statt alle Transaktionen zu importieren, kann eine auf Tages-, Wochen- oder Monatsbasis zusammengefasste Tabelle verwendet werden:

- Aggregation auf Tagesebene kann ein Modell um 90 % verkleinern
- Aggregation auf Monatsebene kann es um bis zu 99 % verkleinern

Der Kompromiss: Drilldown auf Transaktionsebene ist dann nicht mehr möglich. Diesen Konflikt lässt sich mit einem zusammengesetzten Modell (Import für aggregierte Daten + DirectQuery für Detailseiten) entschärfen.

DirectQuery-Modelle optimieren

DirectQuery überträgt alle Abfragen direkt an die Quelldatenbank. Die Leistung hängt daher stark von der Datenbank selbst ab — sowie von Netzwerklatenz und Serverauslastung.

Wann DirectQuery sinnvoll ist

- Häufige Datenänderungen und Quasi-Echtzeitanforderungen
- Sehr große Datenmengen, die nicht sinnvoll importiert werden können
- Anforderungen an Datenhoheit (Daten dürfen das System nicht verlassen)
- Mehrdimensionale Quellen (z. B. SAP BW mit eigenen Measures)

Verhalten von DirectQuery-Verbindungen

Beim Laden werden **keine Daten** importiert — nur das Schema. Bei jeder Visual-Interaktion werden Abfragen an die Quelle gesendet. Wenn die Quelldaten sich ändern, werden vorhandene Visuals **nicht automatisch aktualisiert** — eine manuelle Aktualisierung ist notwendig. Im Power BI-Dienst veröffentlichte DirectQuery-Datasets enthalten keine Daten; bei jedem Berichtsaufruf wird die Quelldatenbank live abgefragt.

Einschränkungen

- Leistung direkt abhängig von der Quelldatenbank
- Sicherheit: Alle Benutzer sehen die Daten, auf die Power BI Zugriff hat
- Datentransformationen in Power Query eingeschränkt (kein Pivot/Unpivot bei OLAP-Quellen)
- Einige Modellierungsfunktionen nicht verfügbar

Optimierungsmaßnahmen

In der Quelldatenbank (größte Wirkung):

- Indizes überprüfen und optimieren
- Komplexe berechnete Spalten auf Datenbankebene auslagern (statt in Power BI)
- Surrogatschlüsselspalten in Dimensionstabellen hinzufügen

In Power BI Desktop:

- Leistungsanalyse für Abfrageüberwachung nutzen
- Anzahl der Visuals und Felder pro Visual reduzieren

- Unnötige Spalten und Zeilen entfernen

Abfragereduzierungsoptionen (Datei → Optionen → Abfragereduzierung):

Option	Wirkung
Kreuzfilter-Interaktionen deaktivieren	Visuals interagieren nicht mehr standardmäßig miteinander
Datenschnitte: "Anwenden"-Schaltfläche	Änderungen werden erst bei Klick auf "Anwenden" gesendet
Filter: "Anwenden"-Schaltfläche	Filteränderungen werden manuell bestätigt

Diese Optionen reduzieren die Anzahl der Abfragen an die Datenquelle und vermeiden ständige Aktualisierungen bei jeder Benutzerinteraktion.

Aggregationen erstellen und verwalten

Aggregationen reduzieren Tabellen auf einem höheren Granularitätsniveau — z. B. von Transaktionszeilen auf Tageswerte — und verbessern damit Leistung und Dateigröße erheblich.

Wann Aggregationen sinnvoll sind

- Große Datenmengen, bei denen Übersichtsabfragen die häufigsten sind
- Langsame Aktualisierungszeiten durch zu viele Zeilen
- Modell wächst und soll zukunftssicher sein
- Dateigröße muss reduziert werden (z. B. 1 GB → 50 MB)

Aggregationen in Power Query erstellen

Aggregationen können direkt im Power Query-Editor erstellt werden:

1. **Spalten auswählen** — nur die für die Aggregation notwendigen Spalten behalten
2. **Gruppieren nach** — Spalte(n) auswählen, nach denen gruppiert wird (z. B. **OrderDate**)

3. **Aggregationen hinzufügen** — für jede numerische Spalte den Aggregationstyp festlegen (Summe, Anzahl, Min, Max usw.)

4. **Schließen und übernehmen**

Screenshot Dateiname: 6-aggregate-group-ssm.png — Power Query-Fenster "Gruppieren nach" mit einer Gruppierungsspalte (OrderDate) und mehreren hinzugefügten Aggregationsspalten.

Beispiel Eine Verkaufstabelle mit 999+ Zeilen (eine Zeile pro Auftragsposition) wird auf Tagesebene aggregiert — Ergebnis: 30 Zeilen. Die Modellgröße sinkt drastisch, Abfragen für Übersichtsberichte laufen erheblich schneller.

Screenshot Dateiname: 6-aggregate-data-after-less-rows-ssm.png — Power Query-Editor nach der Aggregation: Zeilenanzahl auf 30 reduziert, gegenüber 999+ vorher.

Aggregationen verwalten

Über **Rechtsklick auf Tabelle im Datenbereich** → **Aggregationen verwalten** lassen sich bestehende Aggregationen anpassen: Zusammenfassungstyp ändern, Detailtabelle oder -spalte anpassen. Änderungen mit **Alle übernehmen** bestätigen.

Zusammenfassung

Leistungsanalyse

Caches leeren, dann Aufzeichnung starten. Drei Kategorien: DAX-Abfrage, Visualanzeige, Andere. DAX > 120 ms genauer untersuchen. DAX Studio für tiefere Abfrageanalyse. Ergebnisse nach Gesamtdauer sortieren.

Variablen

Jede Variable wird nur einmal ausgewertet — bei wiederholter Nutzung deutlicher Leistungsgewinn. Verbessern Lesbarkeit, vereinfachen Debugging durch temporäres RETURN einer Variablen, ersetzen EARLIER/EARLIEST.

Kardinalität

Hohe Spalten-Kardinalität (viele eindeutige Werte) verschlechtert Leistung. Unnötige Spalten entfernen. Beziehungskardinalität korrekt konfigurieren. Voraggregation auf Tages-/Monatsebene reduziert Modell drastisch.

DirectQuery optimieren

Leistung hängt von Quelldatenbank ab. Indizes optimieren, Transformationen auf DB-Ebene auslagern. In Power BI: Visuals reduzieren, Abfragereduzierungsoptionen nutzen. Leistungsanalyse für Abfrageüberwachung.

Aggregationen

Daten auf höherer Granularität zusammenfassen (Tages-/Monatsebene). In Power Query per "Gruppieren nach" erstellen. Reduziert Zeilen dramatisch — bis zu 99% Modellverkleinerung möglich. Über "Aggregationen verwalten" anpassen.

Auto Datum/Uhrzeit

Erstellt für jede Datumsspalte ausgeblendete berechnete Tabellen. Vergrößert das Modell und verlangsamt Aktualisierungen. Bei eigener Datumstabelle deaktivieren: Optionen → Daten laden → Zeitintelligenz.



Modul 3 – Semantisches Modell & DAX
Visuelle Berechnungen

Modul 4 – Analysen

Berichtsanforderungen bestimmen

